



HOMEWORK 1 REPORT

RECOGNIZING HANDWRITTEN DIGITS BY OPTIMIZATION:
FITTING A REGULARIZED SMOOTHED-HINGE LINEAR CLASSIFIER WITH
FIXED-STEP GRADIENT DESCENT



COURSE:
MATH-329 – CONTINUOUS OPTIMIZATION

GROUP:

STUDENTS:
Tara DAVIES
Noalyne DEKER
Victor HUBERT
Boray KASAP

DATE:
SEPTEMBER 28, 2026

1 Existence and uniqueness

ϕ is differentiable. On each of the intervals $]-\infty, -1]$, $[-1, 0]$ and $[0, +\infty[$, ϕ coincides with a polynomial, so it is differentiable on the corresponding open intervals. At the junction points -1 and 0 , the polynomials on either side agree, and so do their derivatives: each one-sided derivative is that of the polynomial on that side, and at -1 both equal 0 , at 0 both equal 1 . Hence ϕ is differentiable on all of $\mathbb{R}$, with, for all $z \in \mathbb{R}$,

$$\phi'(z) = \begin{cases} 0 & \text{if } z \leq -1, \\ 1 + z & \text{if } -1 \leq z \leq 0, \\ 1 & \text{if } 0 \leq z. \end{cases}$$

ϕ is convex. We use Lemma 4.22. Let $x, y \in \mathbb{R}$. The case $x = y$ is trivial, so suppose $x \neq y$. By the mean value theorem there exists c strictly between x and y with $\phi(y) - \phi(x) = \phi'(c)(y - x)$. Since ϕ' is non-decreasing (see the formula above), $\phi'(c) \geq \phi'(x)$ when $y > x$ and $\phi'(c) \leq \phi'(x)$ when $y < x$; in both cases $\phi'(c)(y - x) \geq \phi'(x)(y - x)$. Hence $\phi(y) \geq \phi(x) + \phi'(x)(y - x)$, so ϕ is convex by Lemma 4.22.

Deduce f_λ strongly convex and differentiable. For each $i \in \llbracket 1, m \rrbracket$, the map $\theta \mapsto \phi(s_i \tilde{x}_i^\top \theta)$ is convex on $\mathcal{E}$, as the composition of the convex function ϕ with an affine map (Exercise 4.17). A non-negative combination of convex functions is convex (Exercise 4.13), so $f(\theta) = \sum_{i=1}^m \phi(s_i \tilde{x}_i^\top \theta)$ is convex. Since $f_\lambda(\theta) - \frac{\lambda}{2} \|\theta\|^2 = f(\theta)$ is convex and $\lambda > 0$, f_λ is λ -strongly convex by definition. Moreover, f_λ is differentiable, as a sum of compositions of the differentiable function ϕ with affine maps and of the quadratic $\theta \mapsto \frac{\lambda}{2} \|\theta\|^2$.

Deduce f_λ has a unique critical point. Since f_λ is strongly convex, it admits exactly one global minimizer $\theta^* \in \mathcal{E}$ (Theorem 4.28). Since f_λ is moreover convex and differentiable, Corollary 4.23 tells us that $\theta \in \mathcal{E}$ is a global minimizer of f_λ if and only if $\nabla f_\lambda(\theta) = 0$. Combining the two, the set of critical points of f_λ is exactly the set of global minimizers, which is the singleton $\{\theta^*\}$.

From here on we fix $\lambda = 0.005$.

2 Objective and gradient

2.1 Derivation

We write $f(\theta) = \sum_{i=1}^m \phi(g_i(\theta))$ and $f_\lambda(\theta) = f(\theta) + h(\theta)$, with $g_i(\theta) = s_i \langle \tilde{x}_i, \theta \rangle$ for $i \in \llbracket 1, m \rrbracket$ and $h(\theta) = \frac{\lambda}{2} \|\theta\|^2$, for $\theta \in \mathcal{E}$.

In what follows, $\theta, v \in \mathcal{E}$ are arbitrary.

We compute ∇h . We have:

$$\begin{aligned} Dh(\theta)[v] &= \lim_{t \rightarrow 0} \frac{h(\theta + tv) - h(\theta)}{t} \\ &= \frac{\lambda}{2} \lim_{t \rightarrow 0} \frac{\langle \theta + tv, \theta + tv \rangle - \langle \theta, \theta \rangle}{t} \\ &= \frac{\lambda}{2} \lim_{t \rightarrow 0} (2\langle \theta, v \rangle + t\langle v, v \rangle) \\ &= \langle \lambda\theta, v \rangle. \end{aligned}$$

Hence $\nabla h(\theta) = \lambda\theta$.

We compute ∇g_i for $i \in \llbracket 1, m \rrbracket$. We have:

$$\begin{aligned} Dg_i(\theta)[v] &= \lim_{t \rightarrow 0} \frac{g_i(\theta + tv) - g_i(\theta)}{t} \\ &= \lim_{t \rightarrow 0} \frac{s_i \langle \tilde{x}_i, \theta + tv \rangle - s_i \langle \tilde{x}_i, \theta \rangle}{t} \\ &= \langle s_i \tilde{x}_i, v \rangle. \end{aligned}$$

Hence $\nabla g_i(\theta) = s_i \tilde{x}_i$.

We compute ∇f . By the chain rule and linearity of the differential,

$$\begin{aligned} Df(\theta)[v] &= \sum_{i=1}^m D\phi(g_i(\theta)) [Dg_i(\theta)[v]] \\ &= \sum_{i=1}^m \phi'(s_i \langle \tilde{x}_i, \theta \rangle) s_i \langle \tilde{x}_i, v \rangle \\ &= \left\langle \sum_{i=1}^m \phi'(s_i \langle \tilde{x}_i, \theta \rangle) s_i \tilde{x}_i, v \right\rangle. \end{aligned}$$

Hence $\nabla f(\theta) = \sum_{i=1}^m \phi'(s_i \langle \tilde{x}_i, \theta \rangle) s_i \tilde{x}_i$.

We compute ∇f_λ . Again by linearity,

$$\nabla f_\lambda(\theta) = \nabla f(\theta) + \nabla h(\theta) = \sum_{i=1}^m \phi'(s_i \langle \tilde{x}_i, \theta \rangle) s_i \tilde{x}_i + \lambda\theta$$

For computations purposes, we will rewrite the gradient to use more of numpy's matrix multiplication. We place ourselves in the computation situation, i.e we consider that $\tilde{x}_i \in \mathbb{R}^{d+1}$, and $\theta \in \mathbb{R}^{d+1}$. We extend ϕ' to $\mathcal{E}$, where we apply the function to each component of the input vector. Let $\tilde{X} = (\tilde{x}_1, \dots, \tilde{x}_m) \in \mathbb{R}^{(d+1) \times m}$, $s = (s_1, \dots, s_m)^\top \in \mathbb{R}^m$. We then have :

$$\begin{aligned} \nabla f_\lambda(\theta) &= \sum_{i=1}^m \phi'(s_i \langle \tilde{x}_i, \theta \rangle) s_i \tilde{x}_i + \lambda\theta \\ &= \tilde{X}(s \odot \phi'(s \odot (\tilde{X}^\top \theta))) + \lambda\theta \end{aligned}$$

2.2 Implementation and verification

We compare the non-vectorized and vectorized implementations on the same random θ , computing their relative difference. We obtain for f_λ and ∇f_λ a relative error of approximately 7.2×10^{-16} and 3.2×10^{-14} respectively. Both errors are at the level of floating-point rounding, which is expected since the two versions sum the m terms in a different order, so the implementations agree up to numerical precision.

We time both versions on the same data, averaging 50 runs timed with Python's `time.perf_counter()`. The loop takes 0.1560s, and the vectorized version 0.0098s, i.e. about $16\times$ faster. This is because the loop redoes the same small computation m times in Python, which is slow, while the vectorized version does the same total amount of work using NumPy's fast, precompiled array operations.

3 Gradient check

We take θ as a random vector (fixed seed, small entries) and v as a random direction normalized to $\|v\| = 1$. The resulting plot is shown in Figure 1; the straight portion has slope ≈ 2 . This supports the correctness of our gradient, since a wrong gradient would generically not produce this slope-2 behavior together with the flattening at small t (floating-point noise floor). It does not prove correctness: the check only compares the gradient against a finite-difference approximation along one direction v , so it could in principle miss an error that happens to vanish along that particular direction.

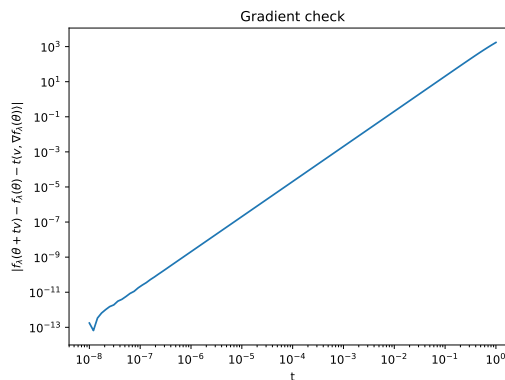


Figure 1: Gradient check: the Taylor residual against the step size t , in log-log coordinates; the linear portion has slope ≈ 2 .

4 Fixed-step gradient descent

We take the constant step size $\alpha = 1/L$, with $L = \sigma_{\max}(X)^2 + \lambda$ as given. Rather than computing an SVD of X (785×12665), we get $\sigma_{\max}(X)^2$ as the largest eigenvalue of $XX^\top$ (785×785), since the eigenvalues of $XX^\top$ are the squared singular values of X , and the smaller matrix is much cheaper to form and diagonalize. This gives $L \approx 7.94 \times 10^5$ and $\alpha = 1/L \approx 1.26 \times 10^{-6}$, which is a standard, easy-to-justify pragmatic choice: it is the largest step size guaranteed safe for an L -smooth objective, so we can use it without tuning.

5 Run

We used a logarithmic vertical axis for both panels, as both quantities vary over several orders of magnitude during the run of fixed-step gradient descent. On a linear scale, the objective curve $f_\lambda(\theta_k)$ would appear almost flat after the first few iterations, the initial drop dominating the vertical range and the following descent becoming visually indistinguishable from a horizontal line. This hides the curvature of the trajectory and makes the transition between the fast initial phase and the slower one unreadable, whereas a logarithmic scale preserves both regimes. The same holds for the gradient norm $\|\nabla f_\lambda(\theta_k)\|$: on a linear plot, the curve would drop sharply and collapse onto zero, making the convergence rate impossible to read.

A linear axis would therefore hide essential structure of both curves, whereas the logarithmic axis reveals the convergence behavior: an early steep descent, followed by a slow phase, and the rate at which the gradient norm approaches zero.

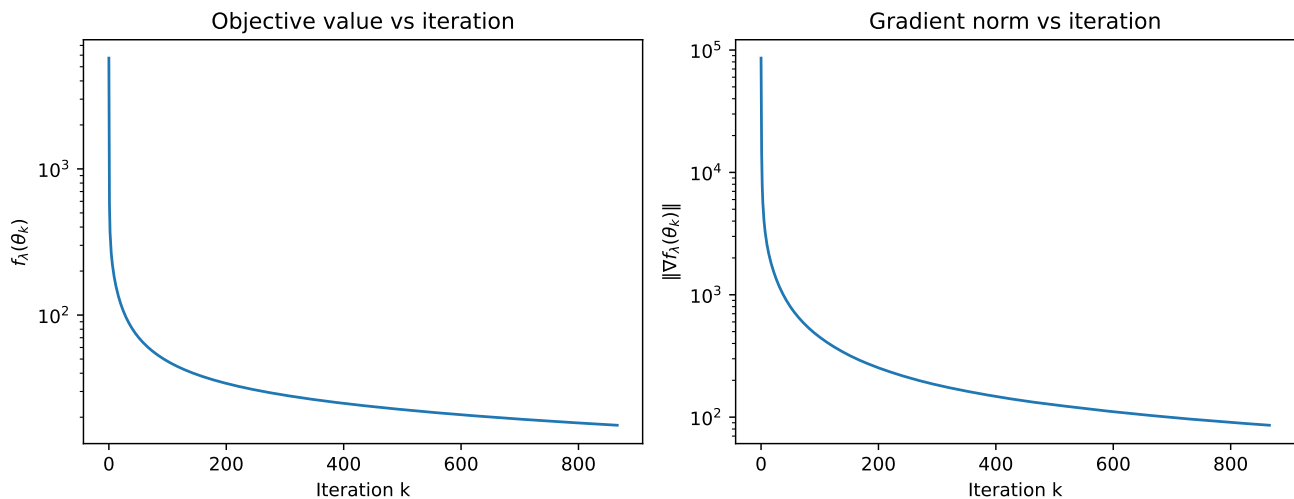


Figure 2: Objective value (left) and gradient norm (right) against the iteration count k for fixed-step gradient descent on f_λ .

6 Interpretation

Strongest applicable result. The sequence is $\theta_{k+1} = \theta_k - \alpha \nabla f_\lambda(\theta_k)$ with a fixed $\alpha > 0$. The strongest result in the lecture notes for such an iteration is Theorem 4.31: if f is μ -strongly convex with L -Lipschitz gradient and $\kappa = L/\mu$, then for any θ_0 , GD with step size $1/L$ satisfies

$$f(\theta_k) - f(\theta^*) \leq \left(1 - \frac{1}{\kappa}\right)^k (f(\theta_0) - f(\theta^*)),$$

i.e. the optimality gap converges at least linearly to 0, from any starting point.

Assumptions and step size. Both hypotheses hold for f_λ : it is μ -strongly convex with $\mu = \lambda = 0.005$ (Question 1), and its gradient is L -Lipschitz with $L = \sigma_{\max}(X)^2 + \lambda$ (given). The theorem is stated for the step size $1/L$ exactly; Question 4 uses $\alpha = 1/L$ with $L = 7.94286 \times 10^5$, i.e. $\alpha = 1.25899 \times 10^{-6}$, so it applies to our choice.

Prediction versus observation. The condition number is $\kappa = L/\mu \approx 1.59 \times 10^8$, so the guaranteed contraction factor per iteration is $1 - 1/\kappa \approx 1 - 6.3 \times 10^{-9}$. The theorem thus predicts that the optimality gap converges linearly to 0, but so slowly that over our $K = 867$ iterations it only guarantees a reduction by a factor $(1 - 1/\kappa)^{866} \approx 0.999995$. The run does far better: f_λ fell from 5730.1 to 17.63, so, since $f^* \geq 0$, the gap $f_\lambda(\theta_k) - f^*$ shrank by a factor of at least $5730.1/17.63 \approx 325$; moreover, both curves in Figure 2 are concave rather than straight, a fast initial decrease that keeps slowing down.

7 Does the classifier work?

Using the final iterate θ_{final} from the run of Question 5, we classify each sample according to the prescribed rule: label 1 when $\tilde{x}_i^\top \theta_{\text{final}} > 0$ and label 0 otherwise. The final row of both `train.X` and `test.X` is already all ones, so no additional constant was appended. The resulting classification error rates are given in Table 1.

	Samples	Misclassified	Error rate
Train	12 665	8	6.32×10^{-4}
Test	2 115	1	4.73×10^{-4}

Table 1: Classification error of θ_{final} .

The test error (2 115 samples) is of the same order of magnitude as the training error (12 665 samples). This indicates that the predictor obtained by fixed-step gradient descent generalizes well: it does not overfit the training data and maintains its performance on unseen samples. The test error is in fact slightly lower than the training error, but this should not be over-interpreted: it corresponds to a single misclassified test image out of 2 115, so a second misclassified image would already double it. The two error rates are therefore indistinguishable given the size of the test set, and the only conclusion we draw is that the classifier performs as well on unseen samples as on the training set.

8 What we learned

On the mathematical side, we learned that the regularization term $\frac{\lambda}{2} \|\theta\|^2$ does a lot of work: without it, f is only convex and may not even have a minimizer, while with it f_λ is λ -strongly convex, so the minimizer exists, is unique, is the only critical point, and gradient descent is guaranteed to converge linearly. On the computational side, the loop and vectorized versions compute the same thing (they agree to 10^{-14}) but the vectorized one is about $15\times$ faster, so the way a formula is implemented matters as much as the formula itself. The gradient check was also useful: it is cheap and catches a wrong gradient before it breaks the rest of the homework, even if it does not prove correctness. Finally, experimentally, the run showed us that Theorem 4.31 is a worst-case bound, not a prediction. With $\kappa \approx 1.6 \times 10^8$ it guarantees almost nothing over 866 iterations, but the objective still dropped by a factor above 300.